ECE-GY 7123 Project 1 Report

Samarth Pusegaonkar

New York University
ssp6681@nyu.edu

Abstract

We design and train a residual network for CIFAR-10 image classification under a hard limit of 5 million trainable parameters, trained from scratch on the 50,000 provided images. Rather than hand-picking a model, we run a staged sweep over three axes, architecture, optimizer, and regularization, using short 15-epoch runs. We then retrain the strongest configurations for 150 epochs. The final model is a three-stage ResNet with 4.62M parameters that concentrates computation at high spatial resolution. Trained with SGD, a one-cycle learning-rate schedule, TrivialAugment, and random erasing, it reaches 97.06% accuracy on a held-out validation split. Retrained on all 50,000 images, it scores 88.4% on the public leaderboard.

Introduction

The Competition

The Efficient ResNet Architecture Challenge asks participants to build the most accurate CIFAR-10 classifier possible with a modified residual network of at most 5 million trainable parameters. CIFAR-10 (Krizhevsky 2009) consists of 32×32 RGB images from ten classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The competition supplies 50,000 labeled training images and 2,000 unlabeled test images. All models must be trained from scratch: pretrained weights and external data are prohibited. Predictions are scored by classification accuracy on a hidden labeled test set, split equally into a public leaderboard, shown during the competition, and a private leaderboard that determines final rankings. The challenge is therefore not to build the largest network, but to decide where a fixed parameter budget is best spent and how to train it.

Residual Networks

ResNets (He et al. 2016) make very deep convolutional networks trainable by wrapping groups of layers in a skip connection:

$$y = \mathrm{ReLU}\big(F(x) + S(x)\big), \tag{1}$$

where F is a learned residual transformation and S is the skip path. The skip path gives gradients a direct route to earlier layers, so adding blocks does not make optimization harder. In the *BasicBlock* used here, F is two $k \times k$ convolutions, each followed by batch normalization (Ioffe and Szegedy 2015), with a ReLU between them. S is the identity when input and output shapes match, and a 1×1 strided convolution with batch normalization (a projection) when the block changes resolution or width. Blocks are grouped into *stages*; the first block of each later stage halves the spatial resolution and usually increases the channel count. A network is then defined by the number of stages, the width of each stage, the number of blocks per stage, and the kernel size k. These are the architectural knobs we tune.

Training Techniques

Optimizers. SGD with Nesterov momentum updates the weights along a running average of past gradients, evaluated at a look-ahead point. Adam (Kingma and Ba 2015) adapts a per-parameter step size from running estimates of the gradient's first and second moments. AdamW (Loshchilov and Hutter 2019) decouples weight decay from that adaptive step, which restores its regularizing effect. RMSProp (Tieleman and Hinton 2012) divides each gradient by a running average of its recent magnitude.

One-cycle learning-rate schedule. The one-cycle policy (Smith and Topin 2019) raises the learning rate from a small value to a peak $\eta_{\max}$ over the first part of training, then anneals it to nearly zero by the final step, cycling momentum in the opposite direction. The high-learning-rate phase acts as a regularizer, and the final annealing phase settles the model into a good minimum. Because the schedule is defined by the total number of steps, a run of any length ends fully annealed.

TrivialAugment. TrivialAugment (Müller and Hutter 2021) applies a single image operation to each training sample. The operation is drawn uniformly from a fixed set, such as rotation, shear, translation, color, contrast, brightness, sharpness, posterize, solarize, and equalize, and applied at a uniformly random strength. Unlike earlier automatic augmentation methods, it requires no search or tuning, yet matches or beats them on CIFAR-10.

Random erasing. Random erasing (Zhong et al. 2020) replaces a randomly sized and placed rectangle of each training image, with probability p, by random values. This forces the network to rely on more than one discriminative region and simulates occlusion.

Label smoothing. Label smoothing (Szegedy et al. 2016) trains against a target that mixes the one-hot label with the uniform distribution: $(1-\epsilon)$ on the true class and $\epsilon/10$ spread over all classes. This discourages over-confident logits. With $\epsilon = 0.1$, the cross-entropy cannot fall below about 0.5, a fact we use to diagnose memorization.

Weight decay and dropout. Weight decay penalizes large weights. Dropout (Srivastava et al. 2014) randomly zeroes activations during training. We apply it only before the final classifier, since batch normalization already regularizes the convolutional layers.

Test-time augmentation. At inference, we average the network's logits over an image and its horizontal mirror.

Methodology

Data and validation. We hold out a fixed random 5,000 of the 50,000 training images (seed 0) for validation and train on the remaining 45,000. Inputs are normalized with the CIFAR-10 per-channel mean and standard deviation. At evaluation we apply horizontal-flip test-time augmentation (TTA), summing the logits of each image and its mirror. With 5,000 validation images, the standard error of an accuracy near 92% is about 0.4 points, so we treat smaller differences between single runs as ties.

Default training recipe. Unless a sweep changes it, every run uses batch size 256, SGD with Nesterov momentum 0.9, peak learning rate 0.1, weight decay 5×10^{-4}, and cross-entropy with label smoothing 0.1. The one-cycle schedule uses PyTorch defaults: 30% warm-up from $\eta_{\max}/25$, cosine annealing to $\eta_{\max}/10^4$, and momentum cycled between 0.85 and 0.95. Augmentation is a random crop from a 4-pixel reflect-padded image, a horizontal flip, TrivialAugment, and random erasing with $p = 0.25$. Each run uses the same seed and split, so differences between runs reflect only the configuration. Training ran in fp32 on an Apple-silicon GPU (PyTorch MPS backend).

Sweep procedure. A full grid over every factor was infeasible on our hardware: one 15-epoch run takes 7–28 minutes. We therefore ran a greedy coordinate search along three axes, in order. Each axis is swept with 15-epoch runs while all other settings stay at the incumbent best. Because the one-cycle schedule completes within each run, short runs are fully annealed rather than truncated. At each stage, the winner is chosen over the union of the current and all previous stages, so a change is adopted only if it beats the incumbent. After the sweep, the two best distinct configurations are retrained for 150 epochs. Every run is appended, with its configuration and per-epoch curves, to a results log. This makes the sweep resumable and every table in this report reproducible from a single notebook.

Axis 1: architecture. We compare six layouts of 4.3–4.9M parameters, each built from the same template: a 3×3 stem convolution, stages of BasicBlocks, global average pooling, optional dropout, and a linear classifier (Table 1). Each layout tests a different way to spend the budget: more depth, more width, more computation early, an extra downsampling stage, or larger kernels.

Layout	Widths	Blocks	k	Params
base	64-128-256	3-4-3	3	4.62M
deep-narrow	48-96-192	6-6-5	3	4.39M
wide-shallow	96-192-320	2-2-2	3	4.90M
early-heavy	96-160-256	4-4-2	3	4.62M
4-stage	48-96-192-288	1-2-2-2	3	4.32M
large-kernel	40-80-160	2-3-3	5	4.58M

Table 1: Architecture candidates. Widths are channels per stage, starting at 32×32 resolution and halving per stage.

Axis 2: optimizer. On the best architecture, we compare SGD with Nesterov momentum at peak learning rates $\{0.05, 0.1, 0.2\}$, AdamW at $\{10^{-3}, 3 \times 10^{-3}\}$ with decoupled weight decay 5×10^{-2}, and RMSProp (momentum 0.9) at $\{3 \times 10^{-4}, 10^{-3}\}$. Each optimizer gets its own learning-rate range, since a single shared value would unfairly favor one of them.

Axis 3: regularization. On the best architecture and optimizer, we vary weight decay ($\times 0.3$ and $\times 3$ the incumbent), add dropout 0.2 before the classifier, and weaken augmentation, either to crop + flip only or to crop + flip + TrivialAugment without random erasing. Label smoothing was held fixed at 0.1 throughout.

Results

Sweep outcomes. Table 2 lists every sweep run.

Architecture: at matched parameter counts, accuracy tracks computation at high resolution. The early-heavy layout, which keeps four wide blocks at both 32×32 and 16×16, wins. The 4-stage model, which places most of its parameters at 4×4, is fastest but worst. A deeper, thinner network and larger kernels, which spend parameters on spatial extent rather than channels, both lose.

Optimizer: SGD with Nesterov momentum is best and improves with a higher peak learning rate. AdamW comes within 0.5 points. RMSProp is unstable under the one-cycle schedule and collapses at the higher rate.

Regularization: weight decay and dropout changes are within noise. Weaker augmentation looks clearly better at 15 epochs.

Long runs reverse the augmentation ranking. We retrained the two best distinct configurations (crop + flip only, and crop + flip + TrivialAugment) for 150 epochs (Table 4, Figure 1). The ranking reverses: TrivialAugment reaches 96.96% against 95.78% for crop + flip. Without TrivialAugment, training loss falls to ≈ 0.50, close to the minimum achievable with label smoothing 0.1, meaning the network has essentially memorized its training set. With TrivialAugment it stays at 0.66. In short runs, strong augmentation mainly slows fitting; its regularizing benefit appears only once the model has enough epochs to overfit without it. Short proxy runs are therefore useful for choosing architecture and optimizer, but augmentation strength must be chosen at the

Axis	Setting	Val (%)	Min
Arch.	early-heavy	**92.10**	24.1
	wide-shallow	91.82	16.3
	base	91.40	14.0
	deep-narrow	90.70	17.0
	large-kernel	90.44	18.8
	4-stage	90.10	7.4
Opt.	SGD, η=0.2	**92.40**	28.1
	SGD, η=0.1	92.10	24.1
	SGD, η=0.05	91.68	24.1
	AdamW, η=3e-3	91.92	26.6
	AdamW, η=1e-3	91.38	25.9
	RMSProp, η=3e-4	88.78	21.7
	RMSProp, η=1e-3	80.20	21.0
Reg.	crop + flip only	**93.82**	20.9
	+ TrivialAug. (no erasing)	92.64	21.3
	incumbent (full aug.)	92.40	28.1
	weight decay $\times 3$	92.28	21.1
	dropout 0.2	92.08	21.3
	weight decay $\times 0.3$	91.82	21.3

Table 2: All 15-epoch sweep runs (validation accuracy with flip TTA, and wall-clock minutes). Each axis starts from the previous axis's winner.

target training length. Because of this, we also trained the full augmentation (with random erasing) for 150 epochs as a follow-up run. It reached 97.06%, slightly above TrivialAugment alone, with a final training loss of 0.70, so random erasing was kept.

Final model and settled choices. The final model is the **early-heavy** ResNet (Table 3) with 4,621,354 trainable parameters, below the 5M limit. The settled choices are:

- **Architecture:** widths 96-160-256, blocks 4-4-2, 3×3 kernels, identity skips with 1×1 projections at stage transitions, and no dropout.
- **Optimizer:** SGD with Nesterov momentum 0.9, one-cycle schedule with peak learning rate 0.2, and batch size 256.
- **Regularization:** weight decay 5×10^{-4} and label smoothing 0.1.
- **Augmentation:** random crop, horizontal flip, and TrivialAugment, and random erasing with $p = 0.25$.
- **Inference:** horizontal-flip TTA.

For submission, the best configuration is retrained on all 50,000 images, with no validation holdout, for 250 epochs. Table 4 reports validation and public-leaderboard accuracy for each long run. The TrivialAugment model (96.96% validation) scored 88.2% on the public leaderboard; the final model scored 88.4%.

Conclusion and Future Directions

Under a fixed parameter budget, where computation happens mattered more than raw depth. A three-stage ResNet with more blocks and width at 32×32 and 16×16 beat deeper, wider-late, larger-kernel, and four-stage alternatives

Stage	Output	Blocks	Params
Stem	$96 \times 32 \times 32$	3×3 conv	2,784
Stage 1	$96 \times 32 \times 32$	4	665,088
Stage 2	$160 \times 16 \times 16$	4	1,769,280
Stage 3	$256 \times 8 \times 8$	2	2,181,632
Head	10	pool + linear	2,570
Total			**4,621,354**

Table 3: Final early-heavy architecture.

Run (150 ep. unless noted)	Val (%)	Loss	LB (%)
crop + flip	95.78	0.502	–
+ TrivialAugment	96.96	0.657	88.2
+ TrivialAug. + erasing	**97.06**	0.700	–
Final, all 50k (250 ep.)	n/a	0.672	88.4

Table 4: Long runs: validation accuracy, final training loss, and public leaderboard (LB) accuracy. The final run trains on the validation images, so it has no validation score.

of the same size. SGD with Nesterov momentum and a one-cycle schedule beat AdamW and RMSProp. Strong augmentation was the most important regularizer, but only at the full training length: 15-epoch proxy runs preferred the weakest augmentation, which lost by 1.2 points at 150 epochs. Our best configuration reaches 97.06% validation accuracy with 4.62M parameters, and its all-data retrain scored 88.4% on the public leaderboard.

Future work: knowledge distillation. We limited this project to architecture, optimizer, and standard regularization. The most direct next step, which the competition permits, is knowledge distillation (Hinton, Vinyals, and Dean 2015). The parameter limit applies only to the submitted model. A much larger teacher, for example a wide ResNet well beyond 5M parameters, or an ensemble of several models trained from scratch on the same data, could supervise our 4.6M-parameter student through its softened output distribution. Furlanello et al. (2018) show that even a teacher of the same size as the student can improve it.

Reproducibility. All code is in a single notebook (`proj.ipynb`) that runs the full sweep, logs each run, and regenerates every table from the log. Code: https://github.com/Samarth-SP/ECE7123Project1.

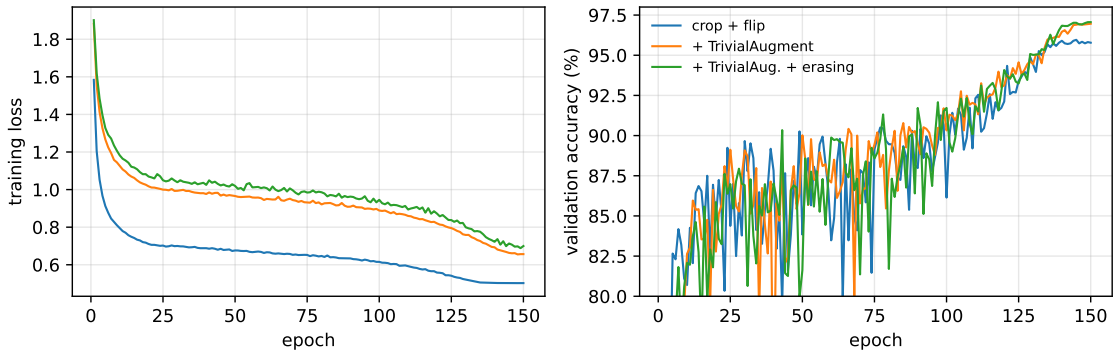


Figure 1: Training loss and validation accuracy of the three 150-epoch runs. Most of the accuracy gain arrives as the one-cycle schedule anneals the learning rate in the final third of training.

References

Furlanello, T.; Lipton, Z. C.; Tschannen, M.; Itti, L.; and Anandkumar, A. 2018. Born-Again Neural Networks. In *International Conference on Machine Learning (ICML)*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531.

Ioffe, S.; and Szegedy, C. 2015. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *International Conference on Machine Learning (ICML)*.

Kingma, D. P.; and Ba, J. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.

Krizhevsky, A. 2009. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto.

Loshchilov, I.; and Hutter, F. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*.

Müller, S. G.; and Hutter, F. 2021. TrivialAugment: Tuning-Free Yet State-of-the-Art Data Augmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Smith, L. N.; and Topin, N. 2019. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*. SPIE.

Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; and Salakhutdinov, R. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15: 1929–1958.

Szegedy, C.; Vanhoucke, V.; Ioffe, S.; Shlens, J.; and Wojna, Z. 2016. Rethinking the Inception Architecture for Computer Vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Tieleman, T.; and Hinton, G. 2012. Lecture 6.5—RMSProp: Divide the Gradient by a Running Average of its Recent Magnitude. COURSERA: Neural Networks for Machine Learning.

Zhong, Z.; Zheng, L.; Kang, G.; Li, S.; and Yang, Y. 2020. Random Erasing Data Augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*.